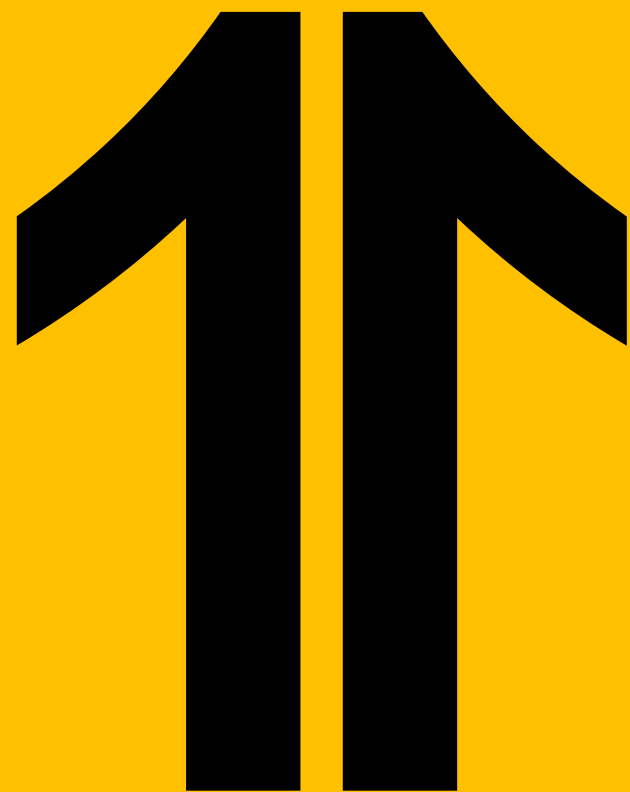




UPN

**UNIVERSIDAD
PRIVADA
DEL NORTE**



EVOLUCIÓN Y CONFIGURACIÓN DE SOFTWARE

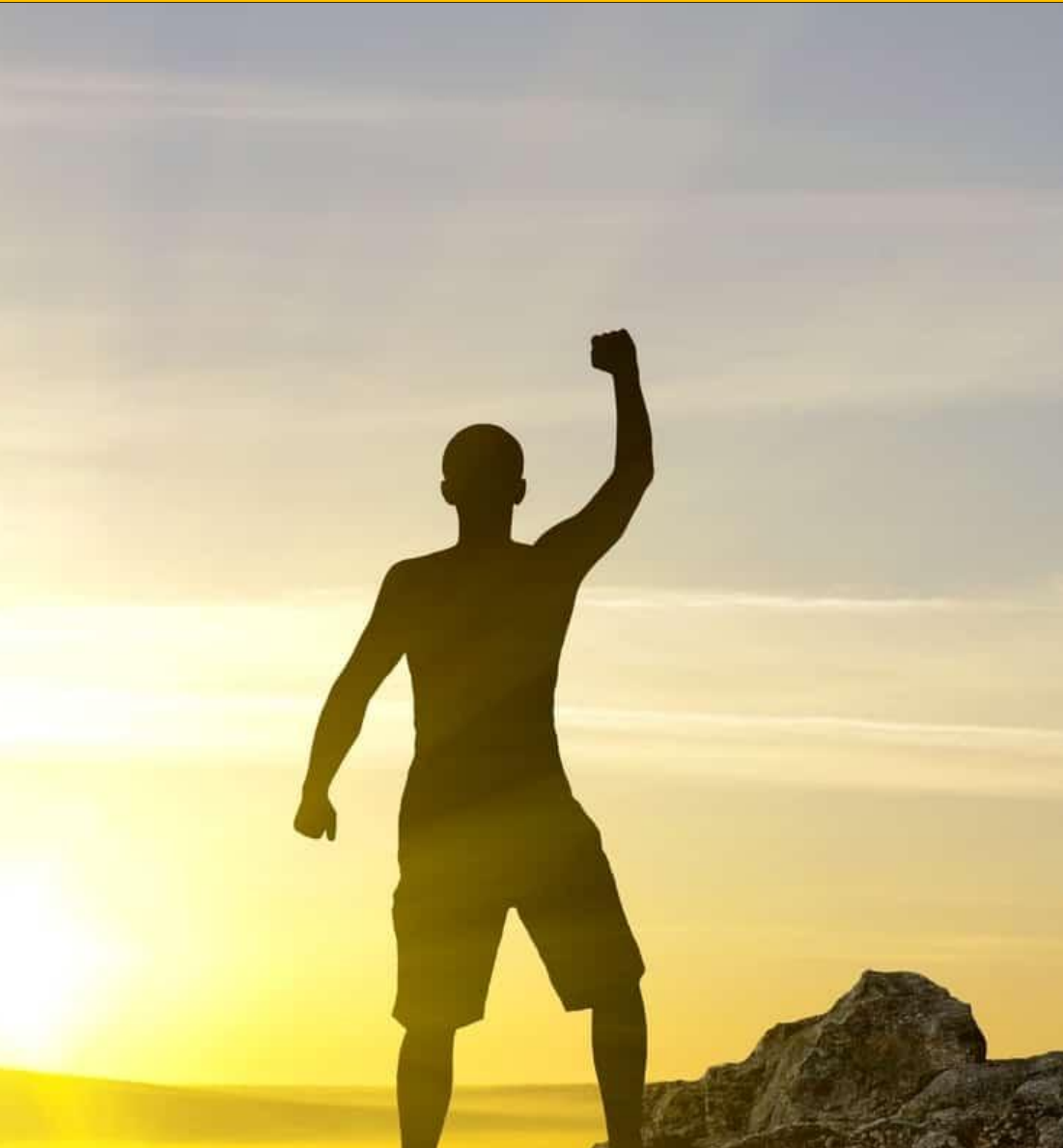
Semana 7

Ing. Edgard Vilchez Valdez.

edgard.vilchez@upn.pe



Logros de la sesión



Al finalizar la sesión, los estudiantes habrán comprendido y aplica la refactorización usando el Principio de Inversión de Control para desacoplar componentes de software, mejorar su mantenibilidad y facilitar su evolución mediante buenas prácticas de diseño.

Índice de la sesión



Índice	Tema
1.1	Refactorización y el Principio Inversión de Control.



Trabajo Grupal: Semana 7



Estimados estudiantes,
Para el próximo trabajo grupal, cada grupo deberá preparar una exposición sobre uno de los siguientes temas. El objetivo es que cada grupo profundice en su tema asignado, presente los conceptos teóricos y prácticos, y los ejemplifique con código funcional.





Objetivo del trabajo

El propósito de este trabajo es que el estudiante comprenda el **Principio de Inversión de Control**, su importancia en el diseño de software mantenible y su aplicación en el proyecto final del curso.

En esta semana, la refactorización solo será utilizada como una acción práctica para mejorar el código aplicando **Inversión de Control**.

Trabajo Grupal: Semana 7



 Investigar los siguientes temas:

Inversión de Control:

- ¿Qué es el Principio de Inversión de Control?
- ¿Qué problema busca resolver en el diseño de software?
- ¿Qué significa que una clase no deba controlar directamente la creación de sus dependencias?
- ¿Cómo ayuda la Inversión de Control a reducir el acoplamiento?
- ¿Qué relación existe entre: Inversión de Control. Inyección de dependencias. Interfaces. Abstracciones.
- ¿Cuál es la diferencia entre depender de una clase concreta y depender de una interfaz?
- ¿Por qué este principio facilita el mantenimiento del software?
- ¿Cómo ayuda a realizar pruebas unitarias?
- ¿Qué beneficios aporta en proyectos que necesitan evolucionar con el tiempo?



Desarrollo Práctico

El grupo debe revisar su proyecto final e identificar una parte del código donde exista una dependencia directa entre clases.

Luego, debe mejorar ese código aplicando el **Principio de Inversión de Control**.

En esta parte, no deben presentar avances generales del proyecto. Solo deben mostrar la mejora relacionada con el tema de la semana.



Desarrollo Práctico

Cada grupo debe identificar un caso donde ocurra alguno de estos problemas:

- Una clase crea directamente un objeto de otra clase usando new.
- Un controlador depende directamente de una clase concreta.
- Un servicio depende directamente de otro servicio específico.
- Una clase usa directamente una implementación concreta en lugar de una interfaz.
- El código sería difícil de probar porque sus dependencias están fijas.
- El cambio de una implementación obligaría a modificar varias clases.
- Existe alto acoplamiento entre módulos del proyecto.

RÚBRICA PARA CALIFICACIÓN INDIVIDUAL (EXPOSICIÓN)



CATEGORÍA	PUNTAJE				MÁXIMO
	5	4-3	2-1	0	
Organización y Estructura de la Presentación	La presentación sigue una estructura lógica y es fácil de seguir.	La estructura es buena pero puede ser mejorada en algunas áreas.	La presentación es confusa o carece de organización.	La presentación no sigue una estructura coherente.	5
Dominio del Tema y Capacidad de Responder Preguntas (Vale el doble)	Demuestra un sólido dominio del tema y responde a todas las preguntas con confianza y precisión.	Muestra un buen conocimiento del tema, aunque con algunas dudas.	Tiene dificultades para responder preguntas, mostrando un dominio limitado del tema.	No pudo responder satisfactoriamente a las preguntas.	10
Calidad Visual y Profesionalismo de la Presentación			La presentación es visualmente atractiva, profesional y sin errores ortográficos o gramaticales.	La presentación es desordenada y carece de profesionalismo.	2
Participación		Entre 81% y 100%	Entre 61% y 41%	<=40%	3
TOTAL					20

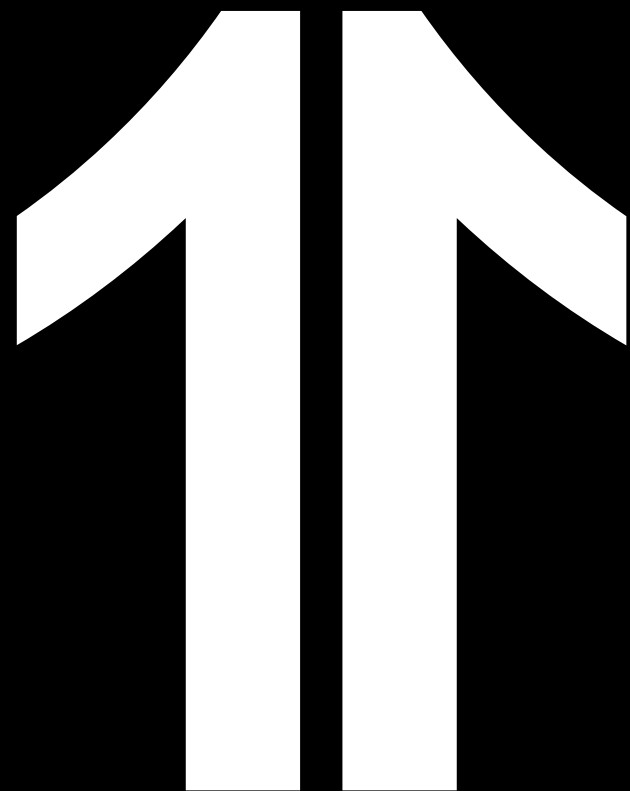


- La refactorización permite mejorar la estructura interna del software sin alterar su funcionamiento externo.
- El Principio Abierto-Cerrado nos enseña a diseñar sistemas que puedan ampliarse sin modificar el código ya probado.
- Aplicar este principio reduce riesgos, facilita el mantenimiento y mejora la escalabilidad del software. Un código bien diseñado es más fácil de entender, reutilizar y adaptar a nuevos requerimientos.
- Pensar en extensibilidad desde el diseño ayuda a desarrollar soluciones más profesionales y sostenibles en el tiempo.



!!! MUCHAS GRACIAS!!!
por su atención...

Ing. Edgard Vilchez Valdez



UPN

**UNIVERSIDAD
PRIVADA
DEL NORTE**